

ORIGINAL ARTICLE OPEN ACCESS

Modeling and Analyzing Urban Networks and Amenities With OSMnx

Geoff Boeing 

University of Southern California, Los Angeles, CA, USA

Correspondence: Geoff Boeing (boeing@usc.edu)**Received:** 12 February 2025 | **Revised:** 8 April 2025 | **Accepted:** 15 April 2025**Keywords:** geospatial data science | open science | open-source software | spatial networks | urban modeling

ABSTRACT

OSMnx is a Python package for downloading, modeling, analyzing, and visualizing urban networks and any other geospatial features from OpenStreetMap data. A large and growing body of literature uses it to conduct scientific studies across the disciplines of geography, urban planning, transport engineering, computer science, and others. The OSMnx project has recently developed and implemented many new features, modeling capabilities, and analytical methods. The package now encompasses substantially more functionality than was previously documented in the literature. This article introduces OSMnx's modern capabilities, usage, and design—in addition to the scientific theory and logic underlying them. It shares lessons learned in geospatial software development and reflects on open science's implications for urban modeling and analysis.

1 | Introduction

Modeling networked urban infrastructure (such as streets, rails, canals, etc.) and geospatial features (such as amenities, buildings, transit stops, etc.) underpins analyses of travel behavior, accessibility, public health, sustainability, and spatial equity. This research spans the disciplines of urban morphology (e.g., Gervasoni et al. 2017; Nilsson and Gil 2019; Coutrot et al. 2022), transport planning (e.g., Merchán et al. 2020; Liao et al. 2020; Natera Orozco et al. 2020), and network science (e.g., Feng and Porter 2020; Yin et al. 2020; Young and Eccles 2020). However, traditional limitations of data availability, inconsistent digitization standards, and a lack of well-documented, reusable tools have historically limited the reproducibility, generalizability, scalability, and usefulness of empirical urban network science (Liu et al. 2022).

The open science movement has arisen in response to similar challenges across many disciplines. Open science seeks

to make scientific processes and results transparent, interpretable, reproducible, and accessible to the public (Kedron et al. 2021). Open-source software is central to this movement. Developing, maintaining, and sharing well-documented, reusable, theory-infused analytical tools in an open commons prevents others from having to reinvent the wheel and accelerates broader scientific endeavors (Singleton et al. 2016). It also makes explicit the innumerable small modeling decisions and underlying assumptions encountered in all research (Gahegan 2018; Rey et al. 2022). Geographers have long operated at the forefront of open science (Rey 2009; Harris et al. 2017). Today, a robust open-source geospatial data science ecosystem exists to support the discipline's technical needs in representing and modeling geospatial data (Fleischmann 2019; Rey 2019; Van den Bossche et al. 2024).

Like their peers across disciplines, urban scholars need good tools for good science, including spatial network science. This article discusses OSMnx, a free, open-source, and fully type-annotated

This is an open access article under the terms of the [Creative Commons Attribution-NonCommercial-NoDerivs](https://creativecommons.org/licenses/by-nc-nd/4.0/) License, which permits use and distribution in any medium, provided the original work is properly cited, the use is non-commercial and no modifications or adaptations are made.

© 2025 The Author(s). *Geographical Analysis* published by Wiley Periodicals LLC on behalf of The Ohio State University.

package written in pure Python that allows users to download, model, analyze, and visualize urban networks and geospatial features from OpenStreetMap data. Users can model walking, driving, biking, or customized networks with a single line of code and just as easily work with urban amenities and points of interest, building footprints, transit stops, elevation data, street orientations, speed, travel time, and routing. This package was first released in beta form in 2016, and that inchoate version was documented in a preliminary article (Boeing 2017). At the time, it was the first tool enabling users to automatically download and model street network geometry and topology for any city in the world as a graph and then analyze and visualize it (Boeing 2020b). It has since become a widespread tool for a broad range of urban modeling and analysis in the intervening years (e.g., Coutrot et al. 2022; Nilsson and Gil 2019; Feng and Porter 2020; Gervasoni et al. 2017; Liao et al. 2020; Natera Orozco et al. 2020; Yin et al. 2020; Young and Eccles 2020).

However, since its initial release, the package has evolved drastically by expanding its functionality, improving performance, and stabilizing a user-friendly API. To document these start-of-the-art capabilities and their scientific underpinnings in the literature, this article presents OSMnx's current modular structure, functionality, and applications for urban analytics. The following section introduces the theory, data, and tools of street network modeling and analysis, including OSMnx's place in that landscape. Then it presents the package's modular organization, capabilities, and usage. Finally, this article concludes with a discussion of lessons learned for geospatial software development and the implications of open geographical science.

2 | Street Network Models and Analysis

To describe precisely what this package does and how it works, we must first introduce some necessary theory underlying street network modeling. Innumerable scientific books and papers cover spatial network theory and analysis (e.g., Tinkler 1979; Barnes and Harary 1983; Gastner and Newman 2006; Barthelemy 2011; Ducruet and Beauguette 2014; O'Sullivan 2014; Marshall et al. 2018). The following subsections discuss the fundamentals of spatial network models relevant to streets, and then briefly review the current landscape of analytics and tools.

2.1 | Model Fundamentals

Real-world networks are commonly modeled as mathematical *graphs* (Trudeau 1994; Barthelemy 2022). A graph, G , is a data structure consisting of two sets: One set, N , contains *nodes* that are linked to each other by the second set, E , which contains node pairs (as sets or tuples) called *edges*. Let $G = (N, E)$ and $\{u, v\} \subseteq N$ and $\{u, v\} \in E$. We can thus say for graph G that: (1) edge $\{u, v\}$ links nodes u and v , (2) edge $\{u, v\}$ is *incident* to u and to v , (3) u is *adjacent* to v and vice versa, and (4) u and v are *neighbors* (Newman 2010). An adjacency matrix can fully represent a graph by defining these adjacent node pairs.

A node's *degree* is the number of edges incident to that node. For example, a node with degree 2 has two incident edges, which are

consequently adjacent to each other. An edge can be *undirected* (a 2-set, thus linking two nodes bidirectionally), *directed* (a 2-tuple, thus linking one-way from a source node to a target node), or a *self-loop* (linking one node back to itself). A graph with directed edges is called a directed graph, or *digraph*. A graph that allows multiple edges to link a single pair of nodes is a *multigraph*, and those multiple edges are called *parallel* edges.

A *path* is a sequence of edges linking a sequence of nodes. The graph *distance* between two nodes is the count of edges in the shortest such path between them. A *weighted* graph's distance is the sum of some edge impedance attribute (e.g., length or time) along the shortest path minimizing that sum. A directed graph is strongly *connected* if a path exists between each ordered pair of nodes, and it is weakly connected if such a path exists only if its edges are undirected. A disconnected graph contains multiple strongly or weakly connected *components*—each a disjoint set of nodes forming its own internally connected subgraph.

A spatial network's nodes and/or edges are embedded in space (Barthelemy 2022). Spatial graphs thus model both topology and geometry (O'Sullivan 2014). *Topology* refers to the structure and configuration of the nodes and edges, whereas *geometry* encompasses positions, lengths, angles, etc. A *planar* graph's topology can be represented in a two-dimensional plane such that its edges intersect only at nodes (Barthelemy and Flammini 2008). Most street networks are nonplanar due to the occasional presence of overpasses and underpasses, but their spatial embedding and “approximate” planarity constrain their topological characteristics relative to other kinds of complex networks (Boeing 2020a). A *primal* graph models a street network's intersections and dead-ends as nodes and its street segments as edges (Porta et al. 2006b). A *line* graph (sometimes called a *dual* graph) inverts this topology to instead model streets as nodes and their intersections as edges (Porta et al. 2006a), though it discards much of the network's geometry (Ratti 2004).

2.2 | Street Network Analysis

Graphs are commonly used for street network analysis, and researchers have applied many geometric measures of spatial networks to street network graphs (Barthelemy 2011). Intersection density (i.e., the count of nodes with degree > 1 , normalized by network area) is perhaps the most common such measure of network “grain” in transport planning and urban design (e.g., Ewing and Cervero 2010). The average street segment length (i.e., mean edge length) offers a linear proxy of block size. Street density is the sum of edge lengths normalized by network area. Circuitry can take on different interpretations, including average circuitry: The sum of edge lengths divided by the sum of great-circle distances between adjacent node pairs, representing the inverse of network edge straightness (Boeing 2019). Note that many such analyses should use undirected graphs to avoid double-counting bidirectional streets relative to one-ways.

Additionally, topological measures from graph theory are often applied to street networks (Barthelemy 2022). The *average node degree* (i.e., mean number of edges incident to the nodes) indicates graph connectedness¹ and is perhaps the most common

topological measure in transport planning and urban design (e.g., Barrington-Leigh and Millard-Ball 2015, 2017a, 2020). Networks with high connectedness can be more robust against perturbation as they offer alternate routing options if parts of the network fail (Boeing and Ha 2024). Various measures of *centrality* are also common (Crucitti et al. 2006). For example, a node's *betweenness centrality* measures the share of all graph shortest paths that pass through the node (Barthelemy 2004; Barthelemy et al. 2013). The graph's maximum betweenness centrality indicates the share of shortest paths that rely on its most important node: High values suggest possible chokepoints that represent single points of failure, such as a bridge connecting a city's halves across a river (Boeing and Ha 2024).

In street network analysis, researchers use these geometric and topological measures to characterize a network's form. Such analyses often also employ path solving (Miller 1999; Wang et al. 2020a). For example, accessibility analyses solve shortest paths (often weighted by length or travel time) from origin nodes (e.g., homes) to destination nodes representing the locations of amenities (e.g., workplaces, schools, transit stops, parks, grocers, hospitals, etc.) to measure access (Foti 2014; Liu et al. 2022). Disaster analyses often simulate emergency responses or evacuations along the network to understand traffic flows following a disaster (Sasabe et al. 2020; Tamakloe et al. 2021). These kinds of analyses can reveal different outcomes for different communities or locations within the city, offering guidance to target practitioners' interventions.

2.3 | Street Network Tools and Data

Various tools exist to model and analyze spatial networks like street networks. For example, ESRI's ArcGIS software includes a Network Analyst extension, and QGIS offers plug-ins for network analysis. However, such GIS tools' network data ingestion and analysis capabilities are usually fairly limited. Conversely, dedicated network analysis tools such as Gephi, IGraph, and NetworkX offer robust network analysis functionality but limited or no geospatial capabilities. Other dedicated spatial network tools exist, often for specific analytical purposes, including PySAL spaghetti for network inference (Gaboardi et al. 2021; Rey et al. 2022), Pandana for urban accessibility (Foti 2014), momcpy for urban morphology (Fleischmann 2019), and stplanr for transport planning (Lovelace and Ellison 2019).

Street network data can come from various sources, including governmental sources like the US Census Bureau's TIGER/Line ShapeFiles, which represent network geometry but lack sufficient topological information to build a properly nonplanar model (Boeing 2025). Alternatively, OpenStreetMap offers a worldwide, public web mapping platform and geospatial database that anyone can contribute to, with some editorial oversight, and includes streets, highways, transit, buildings, footpaths, cycleways, points of interest, and political boundaries (Jokar Arsanjani et al. 2015). Although its coverage varies, it includes network geometry and topology, and its data quality is generally high—particularly so in urban areas, but with notable exceptions in parts of China and sub-Saharan Africa (Barron et al. 2014; Barrington-Leigh and Millard-Ball 2017b).

OpenStreetMap's data model comprises three *element* types: *Nodes* (points), *ways* (i.e., sequences of nodes, which can be either open ways representing lines or closed ways representing polygons), and *relations* (i.e., between nodes and/or ways). These elements can possess one or more *tags*: Key-value pairs containing attribute data. Geometry is defined by nodes' latitude and longitude coordinates, and OpenStreetMap provides a one-dimensional node-edge topology. However, it does not provide this neatly in the graph theory sense. Rather, every OpenStreetMap way element is formally composed of an n -tuple of OpenStreetMap node elements, plus attribute data. Yet these relationships are topological: They are preserved under continuous deformations and transformations of the underlying space.

OpenStreetMap data are available for download from third-party services, such as Geofabrik, and web APIs including Overpass and Nominatim. Historically, however, building a street network model from raw OpenStreetMap data was fairly difficult. For example, the Overpass API can be cumbersome to extract the right data to construct an appropriate spatial graph. Analysts usually had to write extensive ad hoc code to process the raw data into a useful model and then write more code to conduct analyses. Their decisions on how to handle ubiquitous street network characteristics—such as directedness, planarity, self-loops, parallel edges, or culs-de-sac—frequently went undocumented or even unconsidered. With every research team writing its own ad hoc code, innumerable small modeling decisions inevitably would go unreported in the resulting peer-reviewed literature—but these small decisions add up to inhibit interpretability and replicability.

3 | The OSMnx Package

3.1 | Overview

OSMnx (pronounced as the initialism: “oh-ess-em-en-ex”) primarily fits into this landscape by enabling users to build a graph model of a spatial network anywhere in the world by automatically downloading raw data from OpenStreetMap and constructing a spatialized NetworkX model. This allows users to avoid unnecessary ad hoc coding and replaces myriad small modeling decisions with a well-documented, transparent, common tool.

In turn, OSMnx has recently become a standard tool in the literature for both retrieving urban data broadly and for modeling street networks specifically. For example, it has been used by researchers, students, and practitioners across a wide variety of fields, including transport planning (e.g., Natera Orozco et al. 2020; Dumedah and Eshun 2020), vehicle-sharing systems (e.g., Luo et al. 2020; Zhang et al. 2019), urban centrality modeling (e.g., Wang et al. 2020a; Torres et al. 2019), computer science (e.g., Yin et al. 2020; Young and Eccles 2020), persistent homology (e.g., Feng and Porter 2020; Abdelkader et al. 2018), synthetic population modeling (e.g., Tuccillo and Gaboardi 2022, 2023), calculating urban indicators (e.g., Quistberg et al. 2019; Wang et al. 2020b), and trip simulation (e.g., Hernández-Hernández et al. 2019; Merchán et al. 2020).

3.2 | Modular Organization

Here we provide an up-to-date² description of the package's modular organization, capabilities, and usage. OSMnx is structured as a collection of 17 modules that organize related functionality (Table 1). It uses the data structures of NetworkX (Hagberg et al. 2008), a network analysis Python package, and GeoPandas (Van den Bossche et al. 2024), a Python package for working with

TABLE 1 | The OSMnx v2.0 public API's modules and the functionality they expose.

Module	Functionality
bearing	Calculate graph edge compass bearings and orientation entropy.
convert	Convert a graph to/from different data structures.
distance	Calculate spatial distances and find nearest graph node/edge(s) to point(s).
elevation	Attach node elevations from raster files or a Google Maps compatible elevation API, and calculate edge grades.
features	Download OSM geospatial features' geometries and attributes, such as points of interest, building footprints, transit stops, etc.
geocoder	Geocode place names or addresses or retrieve OSM elements by place name or ID, via the Nominatim API.
graph	Download and create graphs from OSM data, using filters to query the Overpass API for built-in network types or a custom filter.
io	Save/load graphs to/from GraphML, GeoPackage, or OSM XML files.
plot	Visualize street networks, routes, orientations, and geospatial features.
projection	Project a spatial graph to a different coordinate reference system.
routing	Calculate graph edge speeds, travel times, and weighted shortest paths between nodes.
settings	Configure global package settings.
simplification	Simplify and consolidate spatial graph nodes and edges.
stats	Calculate geometric and topological network measures.
truncate	Truncate a spatial graph by distance, bounding box, or polygon.
utils	General utility functions.
utils_geo	Miscellaneous geospatial utility functions.

geospatial dataframes. It interacts with three public web APIs to collect data: The OpenStreetMap Nominatim API, the Overpass API, and the Google Maps Elevation API (or their equivalents with the same interface).

3.2.1 | Geocoding and Querying

OSMnx geocodes place names and addresses via the Nominatim API. Users can use the `geocoder` module to geocode place names or addresses to latitude-longitude point coordinates. Or, they can retrieve place boundaries or any other OpenStreetMap elements by name or ID. Using the `features` and `graph` modules, as described below, users can download OpenStreetMap data from the Overpass API by latitude-longitude point coordinates, address, bounding box, bounding polygon/multipolygon, or place name (e.g., neighborhood, city, county, etc.).

3.2.2 | Urban Amenities

Using OSMnx's `features` module, users can search for and download any geospatial features (e.g., points of interest, building footprints, grocery stores, schools, public parks, transit stops, etc.) from the OpenStreetMap Overpass API as a GeoPandas `GeoDataFrame` object. This uses OpenStreetMap tags to search for matching elements within the prescribed query area.

3.2.3 | Modeling Networks

Using OSMnx's `graph` module, users can download any spatial network data (such as streets, paths, rail, canals, power lines, etc.) from the Overpass API and model them as a graph. This process uses filters to query the Overpass API: Users can either specify a built-in network type (e.g., bike, drive, walk, all, etc.) to retrieve matching elements, or they can provide their own custom filter written in OverpassQL.

OSMnx models spatial networks as primal, nonplanar, weighted, directed multigraphs with possible self-loops—specifically, these are NetworkX `MultDiGraph` data structures (Hagberg et al. 2008). It models a one-way street as a single directed edge from node u to node v , but a bidirectional street is modeled with two reciprocal directed edges (with identical geometries)—one from u to v and another from v to u —to represent both possible directions of flow. Because these graphs are nonplanar, they correctly model the topology of interchanges, overpasses, and underpasses. That is, edge crossings in a two-dimensional plane are not nodes in an OSMnx model unless they represent true junctions in the three-dimensional real world.

Under the hood, OSMnx does several things to produce the best possible model, many of which aim to attenuate the “artificial perimeter” problem of making a graph model of a real-world network. It initially buffers the query boundary by 500 meters to create a preliminary graph before truncating it to the user's desired query boundary. This ensures accurate streets-per-node counts around the perimeter. As the graph may contain some disconnected components, OSMnx returns the largest weakly connected component by default, though this is configurable. This

parameterization allows users to control which trade-off they prefer: Either (1) retaining everything within the boundary, even if it includes some small disconnected components that inhibit routing, or (2) discarding some elements within the boundary to ensure graph connectivity. OSMnx also automatically simplifies the graph topology as discussed below.

3.2.4 | Graph Simplification

The `simplification` module automatically processes the network's topology from the original raw OpenStreetMap data to ensure that individual nodes represent individual intersections or dead-ends and edges represent the street segments that link them. This graph simplification is of two primary types: Edge simplification and node consolidation. These algorithms are described in detail in Boeing (2025) and summarized here.

Edge simplification, put simply, merges adjacent edges for a better model. It cleans up the graph's topology so that nodes represent intersections or dead-ends and edges represent street segments, as illustrated by Figure 1. This is important because, in OpenStreetMap's raw data, ways comprise sets of straight-line segments between nodes. That is, nodes are also vertices for streets' curved line geometries, not just intersections and dead-ends. By default, OSMnx simplifies this topology by deleting non-intersection/dead-end nodes, merging the edges between them into a new "simplified" edge, and retaining the complete, true edge geometry as an edge attribute. When multiple OpenStreetMap ways are thus merged into a new graph edge, those ways' attribute values can be aggregated into a single value.

Node consolidation, put simply, merges nearby nodes for a better model. This is important because many real-world street networks feature complex intersections and traffic circles, resulting in a cluster of graph nodes where there is really just one true intersection as considered in transport planning or urban design. Similarly, divided roads are often represented by separate centerline edges. The intersection of 2 divided roads thus creates 4 nodes where each edge intersects a perpendicular edge—but these 4 nodes represent a single intersection in the real world. OSMnx

can consolidate such complex intersections into a single node and optionally rebuild the graph's edge topology accordingly. When multiple OpenStreetMap nodes are thus merged into a new graph node, the nodes' attribute values can be aggregated into a single value.

Edge simplification and node consolidation offer several benefits. They produce a more accurate model that better represents the real world, which in turn yields more reliable network measures—for example, by avoiding the overcounting of complex intersections when calculating intersection density, or by ensuring street segment lengths are not underrepresented. Finally, many graph algorithms' time complexity scales with node or edge count. By generating a graph with (often drastically) fewer nodes and edges—yet no loss of accuracy—many algorithms will complete much faster. This matters most when analyzing large urban networks where runtime becomes a major constraint.

3.2.5 | Converting, Projecting, and Saving

OSMnx's `convert` module can convert a NetworkX `Multidigraph` model to a NetworkX `Multigraph` if the user prefers an undirected representation of the network for specific analytical purposes, as discussed in Section 2.2. It can also convert to a NetworkX `Digraph` if the user prefers a directed graph without any parallel edges. OSMnx can also convert a `Multidigraph` to and from node and edge GeoPandas `GeoDataFrame` objects. The resulting nodes `GeoDataFrame` is indexed by OpenStreetMap node ID, and the resulting edges `GeoDataFrame` is multi-indexed by endpoint node u , endpoint node v , and a *key*—to differentiate parallel edges—just as a `Multidigraph` edge is identified by its u, v, key ordered triplet. This also allows users to load arbitrary node and edge ShapeFiles or GeoPackage layers as `GeoDataFrames`, then convert them to a `Multidigraph` for network analysis.

As these models are all spatial graphs, they have coordinate reference system (CRS) metadata. OpenStreetMap's default CRS is EPSG:4326, but users can project a graph to any other CRS using

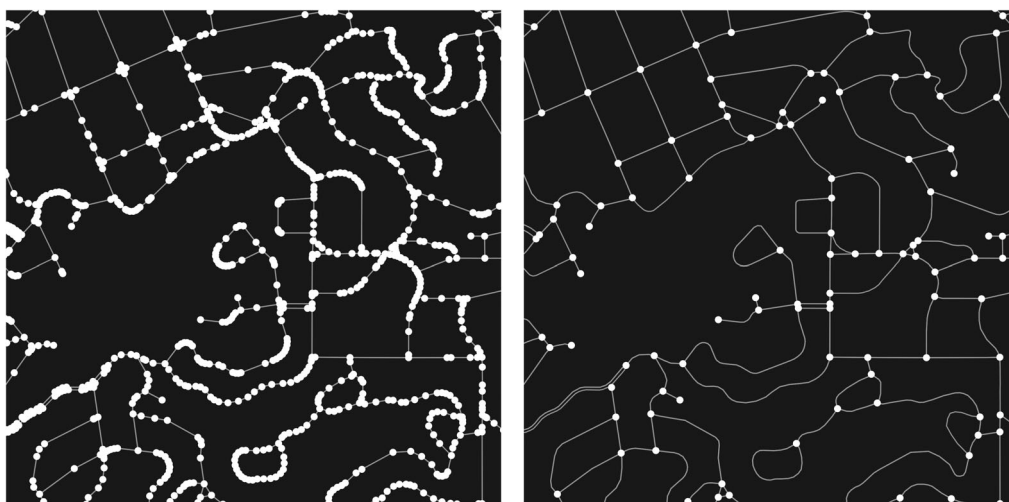


FIGURE 1 | Graph of a town's street network before (left) and after (right) edge simplification. Boeing (2025).

the projection module. If a user is unsure which CRS to project to, OSMnx can automatically determine an appropriate Universal Transverse Mercator CRS for the operation, based on the graph nodes' centroid.

Finally, using the `io` module, users can save a graph to disk as a GraphML file (to load into other network analysis software, including OSMnx), a GeoPackage (to load into other GIS software), or an OSM XML file (the standard OpenStreetMap data interchange format).

3.2.6 | Elevation

Topography is essential to understanding street network form, but street network analyses too often ignore elevation as it can be difficult to acquire and attach to a graph model (Boeing 2022). OSMnx's `elevation` module lets users automatically add elevation attributes to a spatial graph's nodes from either a local raster file or the Google Maps Elevation API. Once all nodes have elevation attributes, users can automatically calculate edge grades (i.e., rise-over-run inclines), analyze the steepness of certain streets, or use elevation change in an impedance function for routing, as discussed below.

3.2.7 | Map Matching and Routing

OSMnx offers basic map matching and routing functionality. The `distance` module can match a list of coordinates to each's nearest node or edge using a fast spatial index and vectorized operation. This can be useful for converting an origin-destination matrix of coordinates (such as geocoded addresses) into corresponding nearest nodes for path solving.

The `routing` module can solve shortest paths for network routing—parallelized with multiprocessing—using different edge weights (e.g., distance, travel time, elevation change, etc.). OpenStreetMap has a “maxspeed” tag representing streets' maximum speed limits, but it tends to be sparse for most cities. To address this problem, the `routing` module can impute missing edge maximum speeds based on observed values across other edges of the same type in the graph. Such imputation can be imprecise, but the user can override it by passing per-type local speed limits. Once all edges have maximum speed attributes, the module can also automatically calculate free-flow traversal times for each edge.

3.2.8 | Spatial Network Measures

The `stats` module can calculate a variety of geometric and topological network measures (Boeing 2022). These measures define streets as the edges in an undirected representation of the graph to prevent double-counting the bidirectional edges of a two-way street. Users can automatically calculate common measures from transport planning, urban design, and network science, including intersection density, circuitry, average node degree, centrality, and many others. Users can also use NetworkX directly to calculate dozens of additional topological network measures. OSMnx's `bearing` module can calculate the streets' compass bearings

and the network's orientation entropy (Boeing 2019). Finally, the `utils_geo` module exposes a set of geospatial utility functions for graph-constrained spatial sampling, point interpolation, and more.

3.2.9 | Visualization

Using OSMnx's `plot` module, users can visualize a wide variety of network characteristics, routes, building footprints, network orientation polar histograms (illustrated by Figure 2 and discussed in Boeing (2019)), and network figure-ground diagrams (illustrated by Figure 3 and discussed in Boeing (2021)). Users can also easily explore networks, routes, or urban amenities as interactive web maps.

3.3 | Installation and Configuration

The OSMnx project's source code³ is publicly hosted on GitHub. Users can install the package from the PyPI package repository or the Anaconda package repository, as detailed in the package documentation's⁴ installation instructions. To get started, users can read the documentation's Getting Started guide, then work through the OSMnx Examples Gallery⁵ for step-by-step tutorials, feature demonstrations, and sample code.

Once installed and imported, users can configure OSMnx through its `settings` module. Here, they can adjust logging behavior, server response caching, server endpoints (including pointing to locally hosted instances), and more. Users can also configure OSMnx to retrieve historical snapshots of OpenStreetMap data as of a certain date.

4 | Lessons Learned in Geospatial Software

Even if OSMnx is good software today, it certainly was not always. The package began life in 2015, and its first version 0 beta release in early 2016 was a single monolithic module that geocoded place queries, downloaded raw OpenStreetMap data, built graph models, and projected graphs to different coordinate reference systems. It was ugly, brittle, and impenetrable—but it worked to pipeline OpenStreetMap data into graph algorithms and analyses in tools like NetworkX or IGraph. The following couple of years saw development continue on two primary fronts: First, in support of specific research needs or grants, with new functions or modules added accordingly. Second, as its user base grew, the project became more community-driven with users submitting bug reports or feature requests and contributing fixes or enhancements to the code base.

This culminated in a major milestone in 2021 with the release of OSMnx version 1.0. In particular, this included several new features and a massive refactoring that entailed some API-breaking changes deemed worthwhile, as its new modular design made the toolkit more comprehensible for new users. Whereas functionality was previously lumped together in poorly architected modules and a monolithic namespace, the new API's design organized functions into thematic (and thus discoverable and predictable) modules.

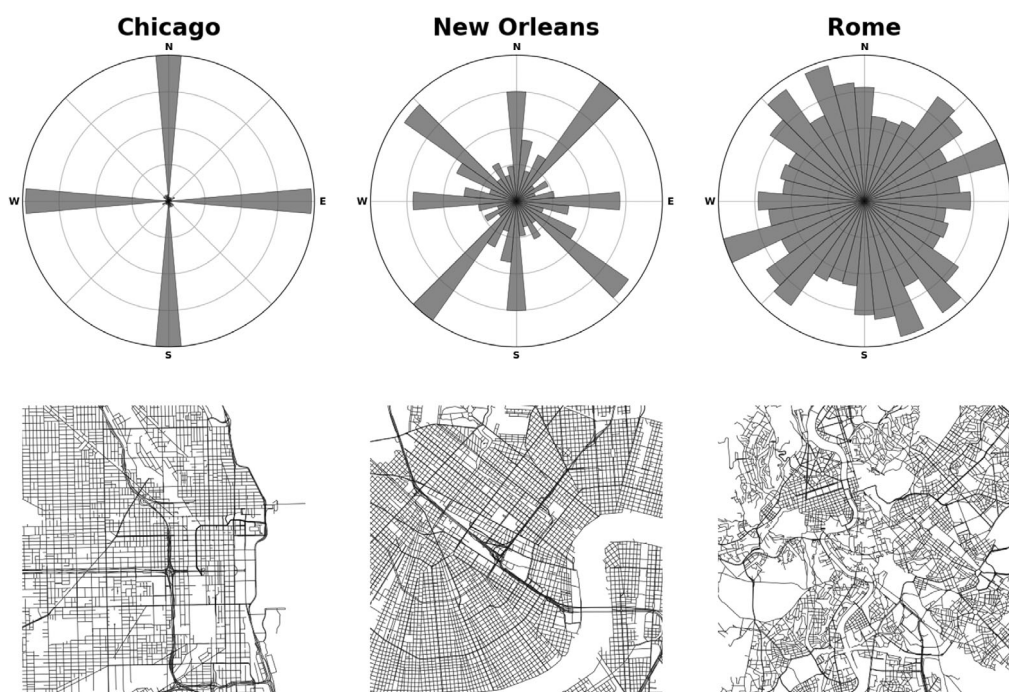


FIGURE 2 | Polar histograms and corresponding street maps illustrating orientation entropy: Low (Chicago), medium (New Orleans), and high (Rome). Histogram bar directions represent street compass bearings, and lengths represent relative frequency. Barthélemy and Boeing (2024).

OSMnx finally reached version 2.0 at the end of 2024, with several new features in addition to under-the-hood best practices aimed at making the package more user-friendly and sustainable for years to come. It is worth considering lessons learned—and suggestions for other researchers’ geospatial software projects—in API design, development pipelines, dependency ecosystems, and the future of open science.

4.1 | API Design

A software package’s API is the interface through which users interact with it. It allows the package to conceal its internals while exposing a stable, well-documented set of functions and parameters. In Python and other languages, these functions are often organized into modules of related functionality.

The OSMnx v2.0 API exposes 17 public modules, as detailed in Section 3.2. Over several years of development, the project has stabilized this API around several principles. It aims for parsimony⁶ and clarity: Functionality is exposed via the fewest necessary functions controlled by the fewest necessary parameters to provide the user an approachable streamlined experience. Such API design is difficult to achieve—especially upfront and particularly in accretionary academic projects—because it requires considerable foresight and software design skill, both of which academics typically only develop along the way. But these design and streamlining efforts in recent years have supported the long-term maintainability of OSMnx’s codebase and promoted the DRY (don’t repeat yourself) principle of software design.

Good documentation is also key to making an API usable and comprehensible. The OSMnx project emphasizes complete and clear documentation, including a “getting started” guide for

newcomers and a comprehensive examples gallery containing tutorials, feature demonstrations, and sample code. Every module and function contains a *docstring* explaining its purpose, important notes or the scientific basis of algorithms and implementations, and the details of parameters and return values. These docstrings make the code “self-documenting” in that their consistent format and contents allow an up-to-date API reference to be automatically rendered and published on the web every time the codebase’s main branch is updated.

Lastly, OSMnx is fully type-annotated. This seemingly inconsequential observation deserves some explanation. One of Python’s benefits is that it is a dynamically-typed language: Its objects and variables do not have their types specified by the coder. Rather, a variable implicitly takes on the data type of whatever value it is assigned, each time it is assigned. The drawback is that bugs can be hard to detect when a coding mistake allows a variable to be assigned a value with an unexpected data type, which then propagates through the system or causes an inscrutable error for edge cases. Type annotation allows the coder to specify objects’ types without enforcing them. This adopts the benefits of type signatures in statically-typed languages like C or Rust by providing (1) type hints to the user and (2) the option of enforcing static typing in the software development pipeline, as discussed below.

4.2 | Development Pipeline

The term “DevOps” refers to integrating software development with deployment and operations processes. OSMnx does this by automating its testing, packaging, and deployment workflows. This includes continuous integration: The practice of maintaining a clean main code branch and frequently merging code changes after they have undergone code review and passed a

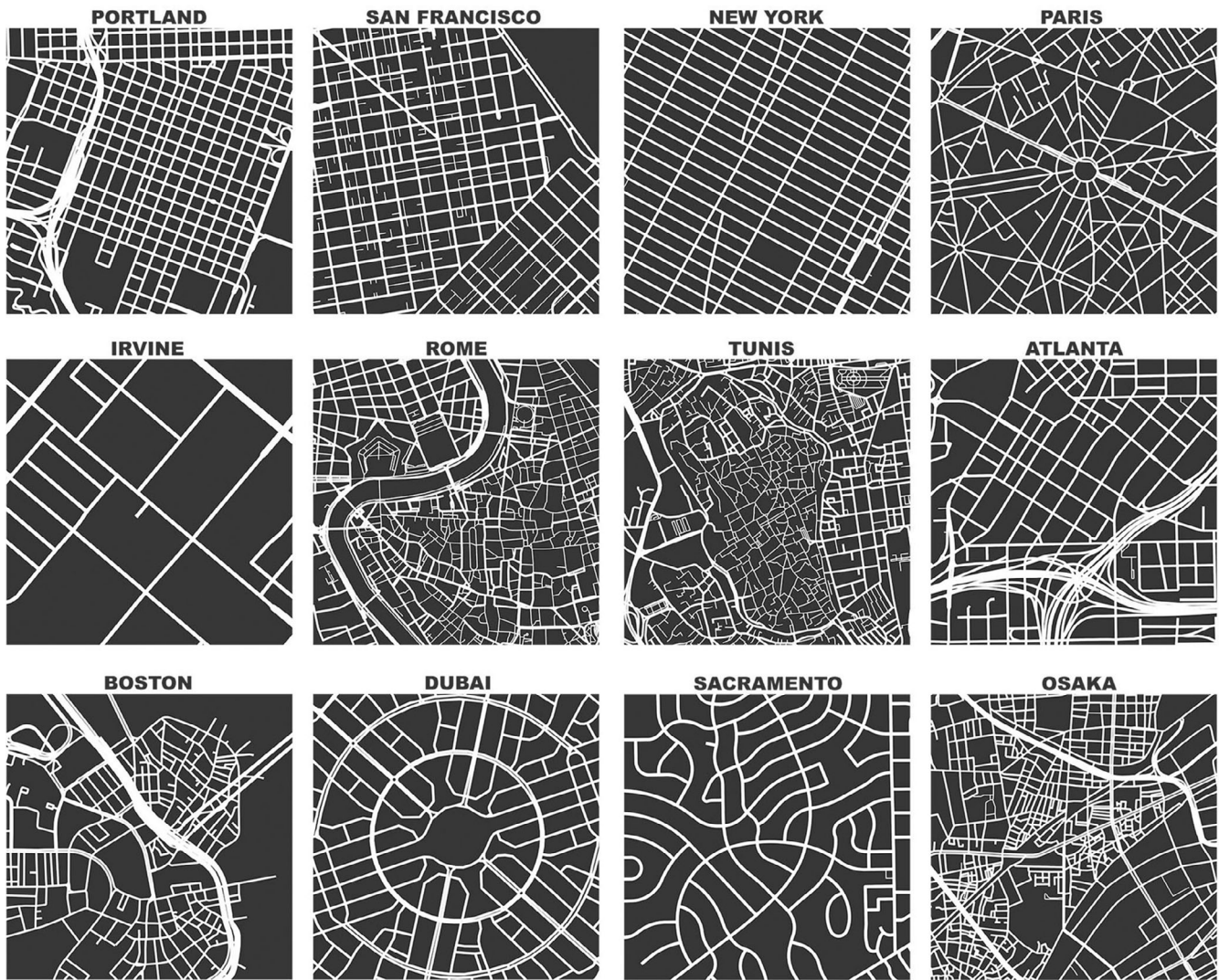


FIGURE 3 | Figure-ground diagrams of one square mile of each city, allowing us to compare street network form in different places. Boeing (2021).

series of tests. Automating these mundane processes is particularly important for relatively small and resource-constrained open-source projects.

The OSMnx project has converged around a modern set of Python-oriented tools to do this. It uses Git for version control and GitHub for project management and collaboration, and its continuous integration test workflow includes (1) a large suite of unit tests run via `pytest`, (2) code coverage tests via `Codecov` to ensure that newly added code is fully tested (OSMnx's current codebase coverage exceeds 98%), (3) code quality, linting, and formatting tests via `Ruff` to ensure that coding best practices and project standards are followed, (4) static type checking via `mypy` and runtime type checking via `Typeguard` to ensure consistent typing throughout the package, and (5) documentation linting and building via `numpydoc` and `Sphinx` to ensure that docstrings meet standards and that the documentation website is always up to date. Some of these standards were decided on and enforced during v2.0 development (requiring significant changes), but it would be much easier to build around them upfront at the start of a project.

Other workflows run on a schedule to test against both the minimum required dependency versions (for backwards compatibility) and pre-release dependency versions (for future compatibility). Finally, two other workflows handle continuous deployment whenever a new release is tagged on GitHub. One builds a Docker image and publishes it to DockerHub, and the other builds the Python package for distribution and publishes it on PyPI for users to install.

4.3 | Dependency Ecosystems

At its inception, the OSMnx project chose the Python programming language because it is general-purpose, easy to learn (reducing barriers to entry for newcomers), and offers a large ecosystem of existing packages providing data structures and functionality. This also means that it can be incorporated into GIS-based workflows via software like ArcGIS and QGIS that use Python for extensibility. In some ways, OSMnx exists as “glue code” in that its models and analyses connect different scientific libraries with scientific data.

The development of OSMnx v2.0 focused on interoperability with the rest of the Python geospatial stack. It depends on six other packages: GeoPandas, NetworkX, NumPy, Pandas, Requests, and Shapely. For extended functionality, it optionally depends on Matplotlib, Rasterio, rio-vrt, SciPy, and scikit-learn. Separating these required vs. optional dependencies allows users to install a lightweight, minimal environment for core functionality or a more extensive environment for complete functionality. The project also reduced its overall number of dependencies without reducing its functionality by streamlining its internals.

OSMnx is tightly coupled to two of its dependencies—GeoPandas (for spatial dataframes) and NetworkX (for graph data structures and algorithms)—as well as OpenStreetMap itself as a data source. Such coupling allows the project to avoid reinventing the wheel by instead offloading the implementation of basic data structures and algorithms to its dependencies. For example, it implements several spatial indices for search routines, including an R-tree (via Shapely), a k -d tree (via Scipy), and a ball tree (via scikit-learn). Reusing existing data structures such as these saves development time and allows each project to specialize in its own contribution without having to innovate everything from scratch.

The future of this ecosystem seems promising. In particular, as an interpreted rather than compiled language, Python has always been hamstrung by relative slowness—but the language's latest releases have made significant speed improvements. Whereas it historically relied on compiled C/C++ extensions for faster execution, the past two years have seen Python increasingly serving as a wrapper around extremely fast Rust code libraries. OSMnx may benefit from this in the future through projects like RustworkX (Treinish et al. 2022), a reimplementation of NetworkX in Rust. Additional speed improvements can be found by shifting linear algebra-based graph algorithms from the CPU to the GPU, as with the cuGraph library. Finally, new data offer new possibilities. Most prominently, Overture Maps now provides worldwide building footprints, roads, political boundaries, addresses, and land cover data—all with a well-structured schema and validation checks. Though new, this initiative deserves geographers' attention.

4.4 | Geospatial Software for Open Science

The academic geography community can advance open science by supporting the open software on which it relies. The first pathway is through educating the next generation on the benefits of open-source solutions and teaching the “whys” and “hows” of algorithms and implementations rather than relying on black-box point-and-click graphical user interfaces. The second pathway is through better incentives and funding. Academic promotion and tenure processes should recognize and reward the scholarly value of open-source research software and open data contributions to both the empirical and theoretical endeavors of geography. Additional funding opportunities for both building and maintaining open research software are key to realizing their positive externalities across communities of scholarship and practice. Finally, the third pathway is through academic publishing. Many GIScience journals are at the forefront of requiring submission of the data and code underlying quantitative manuscripts. This is essential for reproducible and

interpretable science, but it is not enough. We also need our tools to be *reusable*—that is, well-designed, well-documented, well-maintained, generalizable, accessible, and easy to use.

5 | Conclusion

The open science movement calls for open, reusable research tools to support better science. The OSMnx project contributes to these goals by developing and maintaining such a tool for modeling and analyzing urban networks and amenities anywhere in the world from OpenStreetMap data. It embodies the relevant domain theory from urban planning, network science, and geographical information science in a tool designed to “just work.” The OSMnx project improves reproducibility by allowing users to automatically model networks with clearly defined spatial extents, thoroughly documented modeling decisions, and theoretically sophisticated results. For example, its graph models are primal, nonplanar, directed, weighted, multigraphs with possible self-loops to capture the diversity of network types and characteristics around the world. Its graph creation methods attenuate artificial perimeter effects, and its graph simplification methods generate models that flexibly match real-world expectations and use. In turn, researchers need not reinvent the wheel or make dozens of ad hoc basic modeling decisions for each urban network analysis project.

This article documented the OSMnx package's modern organization, capabilities, and theoretical underpinnings. It offered some lessons learned over ten years of geospatial software development and provides suggestions for others working on similar projects. Better tools, led by scholarly experts, represent one of the key paths forward toward better and more open science. Contributing to this shared tools commons benefits everyone across disciplines.

Acknowledgements

The author wishes to thank the developers and maintainers of the many other packages on which the OSMnx project depends, as well as the contributors who have lent their time and expertise in proposing features, solving bugs, and contributing code to the OSMnx code base over the years. Figure 1 is reprinted from Boeing (2025) under the terms of the CC BY-NC-ND 4.0 international license. Figure 2 is reprinted from Barthelemy and Boeing (2024) under the terms of the CC BY 4.0 international license. Figure 3 is reprinted from Boeing (2021) with permission from Elsevier.

Conflicts of Interest

The author declares no conflicts of interest.

Data Availability Statement

Data sharing is not applicable to this article as no new data were created or analyzed in this study.

Endnotes

¹ The separate term *connectivity* has a distinct definition in graph theory, but is less useful for spatial network analysis because approximate planarity sharply constrains it: Almost all real-world street networks have connectivity equal to 1.

² This article documents OSMnx version 2.0.1, released January 2025.

³ The OSMnx source code is hosted at <https://github.com/gboeing/osmnx>.

⁴ The OSMnx documentation is available at <https://osmnx.readthedocs.io/>.

⁵ The OSMnx Examples Gallery is hosted at <https://github.com/gboeing/osmnx-examples>.

⁶ The principle of parsimonious design is perhaps best expressed by Antoine de Saint-Exupéry's famous aphorism: "Perfection is finally attained not when there is no longer anything to add, but when there is no longer anything to take away."

References

- Abdelkader, A., G. Boeing, B. T. Fasy, and D. L. Millman. 2018. "Topological Distance Between Nonplanar Transportation Networks." In *Proceedings of the 28th Fall Workshop on Computational Geometry*, 1–6. City University of New York.
- Barnes, J. A., and F. Harary. 1983. "Graph Theory in Network Analysis." *Social Networks* 5, no. 2: 235–244.
- Barrington-Leigh, C., and A. Millard-Ball. 2015. "A Century of Sprawl in the United States." *Proceedings of the National Academy of Sciences* 112, no. 27: 8244–8249.
- Barrington-Leigh, C., and A. Millard-Ball. 2017a. "More Connected Urban Roads Reduce US GHG Emissions." *Environmental Research Letters* 12, no. 4: 044008.
- Barrington-Leigh, C., and A. Millard-Ball. 2017b. "The World's User-Generated Road Map Is More Than 80% Complete." *PLoS One* 12, no. 8: e0180698.
- Barrington-Leigh, C., and A. Millard-Ball. 2020. "Global Trends Toward Urban Street-Network Sprawl." *Proceedings of the National Academy of Sciences* 117, no. 4: 1941–1950.
- Barron, C., P. Neis, and A. Zipf. 2014. "A Comprehensive Framework for Intrinsic OpenStreetMap Quality Analysis." *Transactions in GIS* 18, no. 6: 877–895.
- Barthelemy, M. 2004. "Betweenness Centrality in Large Complex Networks." *European Physical Journal B: Condensed Matter and Complex Systems* 38, no. 2: 163–168.
- Barthelemy, M. 2011. "Spatial Networks." *Physics Reports* 499, no. 1–3: 1–101.
- Barthelemy, M. 2022. *Spatial Networks: A Complete Introduction*. Springer International.
- Barthelemy, M., and G. Boeing. 2024. "A Review of the Structure of Street Networks." *Findings*. <https://doi.org/10.32866/001c.122117>.
- Barthelemy, M., P. Bordin, H. Berestycki, and M. Griboaudi. 2013. "Self-Organization Versus Top-Down Planning in the Evolution of a City." *Scientific Reports* 3. <https://doi.org/10.1038/srep02153>.
- Barthelemy, M., and A. Flammini. 2008. "Modeling Urban Street Patterns." *Physical Review Letters* 100, no. 13: 138702.
- Boeing, G. 2017. "OSMnx: New Methods for Acquiring, Constructing, Analyzing, and Visualizing Complex Street Networks." *Computers, Environment and Urban Systems* 65: 126–139.
- Boeing, G. 2019. "Urban Spatial Order: Street Network Orientation, Configuration, and Entropy." *Applied Network Science* 4, no. 1: 67.
- Boeing, G. 2020a. "Planarity and Street Network Representation in Urban Form Analysis." *Environment and Planning B: Urban Analytics and City Science* 47, no. 5: 855–869.
- Boeing, G. 2020b. "The Right Tools for the Job: The Case for Spatial Science Tool-Building." *Transactions in GIS* 24, no. 5: 1299–1314.
- Boeing, G. 2021. "Spatial Information and the Legibility of Urban Form: Big Data in Urban Morphology." *International Journal of Information Management* 56: 102013.
- Boeing, G. 2022. "Street Network Models and Indicators for Every Urban Area in the World." *Geographical Analysis* 54, no. 3: 519–535.
- Boeing, G. 2025. "Topological Graph Simplification Solutions to the Street Intersection Miscount Problem." *Transactions in GIS*.
- Boeing, G., and J. Ha. 2024. "Resilient by Design: Simulating Street Network Disruptions Across Every Urban Area in the World." *Transportation Research Part A: Policy and Practice* 182: 104016.
- Coutrot, A., E. Manley, S. Goodroe, et al. 2022. "Entropy of City Street Networks Linked to Future Spatial Navigation Ability." *Nature* 604, no. 7904: 104–110.
- Crucitti, P., V. Latora, and S. Porta. 2006. "Centrality Measures in Spatial Networks of Urban Streets." *Physical Review E* 73, no. 3: 036125.
- Ducruet, C., and L. Beauguitte. 2014. "Spatial Science and Network Science: Review and Outcomes of a Complex Relationship." *Networks and Spatial Economics* 14, no. 3–4: 297–316.
- Dumedah, G., and G. Eshun. 2020. "The Case of Paratransit: Trotro Service Data as a Credible Location Addressing of Road Networks in Ghana." *Journal of Transport Geography* 84: 102688.
- Ewing, R., and R. Cervero. 2010. "Travel and the Built Environment: A Meta-Analysis." *Journal of the American Planning Association* 76, no. 3: 265–294.
- Feng, M., and M. A. Porter. 2020. "Spatial Applications of Topological Data Analysis: Cities, Snowflakes, Random Structures, and Spiders Spinning Under the Influence." *Physical Review Research* 2, no. 3: 033426.
- Fleischmann, M. 2019. "Momepy: Urban Morphology Measuring Toolkit." *Journal of Open Source Software* 4, no. 43: 1807.
- Foti, F. 2014. "Behavioral Framework for Measuring Walkability and Its Impact on Home Values and Residential Location Choices." Dissertation, University of California, Berkeley, CA.
- Gaboardi, J., S. Rey, and S. Lumnitz. 2021. "Spaghetti: Spatial Network Analysis in PySAL." *Journal of Open Source Software* 6, no. 62: 2826.
- Gahegan, M. 2018. "Our GIS Is Too Small." *Canadian Geographer* 62, no. 1: 15–26.
- Gastner, M. T., and M. E. J. Newman. 2006. "The Spatial Structure of Networks." *European Physical Journal B: Condensed Matter and Complex Systems* 49, no. 2: 247–252.
- Gervasoni, L., M. Bosch, S. Fenet, and P. Sturm. 2017. "Calculating Spatial Urban Sprawl Indices Using Open Data." In *Proceedings of the 15th International Conference on Computers in Urban Planning and Urban Management*.
- Hagberg, A. A., D. A. Schult, and P. J. Swart. 2008. "Exploring Network Structure, Dynamics, and Function Using NetworkX." In *Proceedings of the 7th Python in Science Conference*, edited by G. Varoquaux, T. Vaught, and J. Millman, 11–15. SciPy.
- Harris, R., D. O'Sullivan, M. Gahegan, et al. 2017. "More Bark Than Bytes? Reflections on 21+ Years of Geocomputation." *Environment and Planning B: Urban Analytics and City Science* 44, no. 4: 598–617.
- Hernández-Hernández, A. M., J. M. Siqueiros-García, E. Robles-Belmont, and C. Gershenson. 2019. "Anger While Driving in Mexico City." *PLoS One* 14, no. 9: e0223048.
- Jokar Arsanjani, J., A. Zipf, P. Mooney, and M. Helbich. 2015. *OpenStreetMap in GIScience*. Springer International.
- Kedron, P., W. Li, S. Fotheringham, and M. Goodchild. 2021. "Reproducibility and Replicability: Opportunities and Challenges for Geospatial Research." *International Journal of Geographical Information Science* 35, no. 3: 427–445.

- Liao, Y., J. Gil, R. H. M. Pereira, S. Yeh, and V. Verendel. 2020. "Disparities in Travel Times Between car and Transit: Spatiotemporal Patterns in Cities." *Scientific Reports* 10, no. 1: 4056.
- Liu, S., C. Higgs, J. Arundel, et al. 2022. "A Generalized Framework for Measuring Pedestrian Accessibility Around the World Using Open Data." *Geographical Analysis* 54, no. 3: 559–582.
- Lovelace, R., and R. Ellison. 2019. "Stplanr: A Package for Transport Planning." *R Journal* 10, no. 2: 7.
- Luo, M., B. Du, K. Klemmer, H. Zhu, H. Ferhatosmanoglu, and H. Wen. 2020. "D3P: Data-Driven Demand Prediction for Fast Expanding Electric Vehicle Sharing Systems." *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 4, no. 1: 1–21.
- Marshall, S., J. Gil, K. Kropf, M. Tomko, and L. Figueiredo. 2018. "Street Network Studies: From Networks to Models and Their Representations." *Networks and Spatial Economics* 18: 735–749.
- Merchán, D., M. Winkenbach, and A. Snoeck. 2020. "Quantifying the Impact of Urban Road Networks on the Efficiency of Local Trips." *Transportation Research Part A: Policy and Practice* 135: 38–62.
- Miller, H. J. 1999. "Measuring Space-Time Accessibility Benefits Within Transportation Networks: Basic Theory and Computational Procedures." *Geographical Analysis* 31, no. 1: 1–26.
- Natera Orozco, L. G., F. Battiston, G. Iñiguez, and M. Szell. 2020. "Data-Driven Strategies for Optimal Bicycle Network Growth." *Royal Society Open Science* 7, no. 12: 201130.
- Newman, M. E. J. 2010. *Networks: An Introduction*. Oxford University Press.
- Nilsson, L., and J. Gil. 2019. "The Signature of Organic Urban Growth: Degree Distribution Patterns of the City's Street Network Structure." In *The Mathematics of Urban Morphology*, edited by L. D'Acci, 93–121. Springer.
- O'Sullivan, D. 2014. "Spatial Network Analysis." In *Handbook of Regional Science*, edited by M. M. Fischer and P. Nijkamp, 1253–1273. Springer-Verlag.
- Porta, S., P. Crucitti, and V. Latora. 2006a. "The Network Analysis of Urban Streets: A Dual Approach." *Physica A: Statistical Mechanics and its Applications* 369, no. 2: 853–866.
- Porta, S., P. Crucitti, and V. Latora. 2006b. "The Network Analysis of Urban Streets: A Primal Approach." *Environment and Planning. B, Planning & Design* 33, no. 5: 705–725.
- Quistberg, D. A., A. V. Diez Roux, U. Bilal, et al. 2019. "Building a Data Platform for Cross-Country Urban Health Studies: The SALURBAL Study." *Journal of Urban Health* 96, no. 2: 311–337.
- Ratti, C. 2004. "Space Syntax: Some Inconsistencies." *Environment and Planning. B, Planning & Design* 31, no. 4: 487–499.
- Rey, S. J. 2009. "Show Me the Code: Spatial Analysis and Open Source." *Journal of Geographical Systems* 11, no. 2: 191–207.
- Rey, S. J. 2019. "PySAL: The First 10 Years." *Spatial Economic Analysis* 14, no. 3: 273–282.
- Rey, S. J., L. Anselin, P. Amaral, et al. 2022. "The PySAL Ecosystem: Philosophy and Implementation." *Geographical Analysis* 54, no. 3: 467–487.
- Sasabe, M., K. Fujii, and S. Kasahara. 2020. "Road Network Risk Analysis Considering People Flow Under Ordinary and Evacuation Situations." *Environment and Planning B: Urban Analytics and City Science* 47, no. 5: 759–774.
- Singleton, A. D., S. Spielman, and C. Brunsdon. 2016. "Establishing a Framework for Open Geographic Information Science." *International Journal of Geographical Information Science* 30, no. 8: 1507–1521.
- Tamakloe, R., J. Hong, J. Tak, and D. Park. 2021. "Finding Evacuation Routes Using Traffic and Network Structure Information." *Transportation Research Part D: Transport and Environment* 95: 102853.
- Tinkler, K. J. 1979. "Graph Theory." *Progress in Geography* 3, no. 1: 85–116.
- Torres, J. E. H., S. H. González, J. A. J. García, and V. F. Fernández. 2019. "Análisis de Vulnerabilidad de la Infraestructura de Transporte Aplicando Redes Complejas: Red de Avenidas de la Ciudad de Celaya." *Guanajuato. Pistas Educativas* 41, no. 133: 375–391.
- Treinish, M., I. Carvalho, G. Tsilimigkounakis, and N. Sá. 2022. "Rustworkx: A High-Performance Graph Library for python." *Journal of Open Source Software* 7, no. 79: 3968.
- Trudeau, R. J. 1994. *Introduction to Graph Theory*. 2nd ed. Dover Publications.
- Tuccillo, J. V., and J. D. Gaboardi. 2022. *Likeness: A Toolkit for Connecting the Social Fabric of Place to Human Dynamics*. <https://proceedings.scipy.org/articles/majora-212e5952-014>.
- Tuccillo, J. V., and J. D. Gaboardi. 2023. *Spatial Microsimulation and Activity Allocation in Python: An Update on the Likeness Toolkit*. Austin, Texas. <https://proceedings.scipy.org/articles/gerudo-f2bc6f59-00c>.
- Van den Bossche, J., K. Jordahl, M. Fleischmann, et al. 2024. *geopandas/geopandas: v0.14.4*.
- Wang, M., Z. Chen, L. Mu, and X. Zhang. 2020a. "Road Network Structure and Ride-Sharing Accessibility: A Network Science Perspective." *Computers, Environment and Urban Systems* 80: 101430.
- Wang, S., D. Yu, M.-P. Kwan, L. Zheng, H. Miao, and Y. Li. 2020b. "The Impacts of Road Network Density on Motor Vehicle Travel: An Empirical Study of Chinese Cities Based on Network Theory." *Transportation Research Part A: Policy and Practice* 132: 144–156.
- Yin, Y., J. Varadarajan, G. Wang, et al. 2020. "A Multi-Task Learning Framework for Road Attribute Updating via Joint Analysis of Map Data and GPS Traces." In *Proceedings of the Web Conference 2020, WWW '20*, 2662–2668. Association for Computing Machinery.
- Young, D. L., and C. Eccles. 2020. "Automatic Construction of Markov Decision Process Models for Multi-Agent Reinforcement Learning." In *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications II*, vol. 11413, 114130Y. International Society for Optics and Photonics.
- Zhang, Y., D. Lin, and Z. Mi. 2019. "Electric Fence Planning for Dockless Bike-Sharing Services." *Journal of Cleaner Production* 206: 383–393.